

École Hassania des Travaux Publics (EHTP)

Casablanca, Morocco

Engineering Cycle — Computer Engineering

Specification Document

DevSecOps CI/CD Security & Monitoring Platform
SAST, Container Orchestration, SIEM and Incident Response

Prepared by: Amhidi Hamza

Domain: Cybersecurity — DevSecOps — SOC / Incident Response

Version: 1.0

Date: September 2026

Academic year: 2026–2027

Code → Build → SAST → Deploy (Docker/Kubernetes) → Runtime Monitoring (Wazuh) → Log
Analytics (ELK) → Incident Response (TheHive)

Contents

1	Introduction	2
1.1	Context	2
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Scope	3
2	Project Overview	3
3	Requirements Analysis	3
3.1	Actors	3
3.2	Functional Requirements	4
3.3	Non-Functional Requirements	4
4	Technical Architecture	5
4.1	Pipeline and Security Operations Stages	5
4.2	Environments	5
5	Detailed Description of the Modules	5
5.1	Target Application	5
5.2	End-to-End Scenario	6
5.3	Pipeline and Security Modules	6
5.3.1	SAST — Semgrep	6
5.3.2	Code Quality & Security Gate — SonarQube	6
5.3.3	Containerization — Docker	6
5.3.4	Orchestration — Kubernetes	7
5.3.5	Runtime Security Monitoring — Wazuh	7
5.3.6	Log Analytics — Elastic Stack (ELK)	7
5.3.7	Incident Response — TheHive	7
5.4	Alert-to-Case Workflow	7
6	Technology Stack	8
7	Dependencies	8
7.1	Dependencies Between Modules	9
7.2	External Dependencies and Prerequisites	9
8	Planning	9
9	Deliverables	10
10	Validation Criteria and Indicators	10
11	Risks and Mitigation Measures	10
12	Ethical and Legal Constraints	10
13	Conclusion	11
A	Appendix A — Indicative CI/CD Pipeline Skeleton	11
B	Appendix B — Wazuh Alert to TheHive Case (Pseudo-code)	11

1 Introduction

1.1 Context

Modern applications are delivered faster and faster through continuous integration and continuous deployment (CI/CD) pipelines. Every stage of this chain (source code, container image, runtime environment) is a potential attack surface. The *DevSecOps* approach consists in integrating security from the very beginning of the software lifecycle (the *shift-left* approach), while a *Security Operations Center* (SOC) approach ensures that whatever slips through is detected and handled once the application is running.

This project aims to design a platform that covers the **full security lifecycle**: static code analysis before deployment, containerized and orchestrated delivery, and continuous runtime monitoring with structured incident response after deployment.

1.2 Problem Statement

Central question

How can code-level security analysis be automated inside a CI/CD pipeline, and how can the resulting deployment be continuously monitored at runtime so that suspicious activity is detected, logged, correlated and escalated into a trackable incident rather than lost in raw logs?

1.3 Objectives

General objective. Build an automated DevSecOps CI/CD pipeline that integrates security testing, static vulnerability scanning and code-quality analysis, deploys the application through containers orchestrated by Kubernetes, and continuously monitors the running environment to detect and investigate security incidents.

Specific objectives.

- Provide an intentionally vulnerable target application demonstrating realistic application-layer weaknesses.
- Integrate **Semgrep** and **SonarQube** into a CI/CD pipeline for SAST and code-quality/security-quality gating.
- Containerize the application with **Docker** and orchestrate its deployment with **Kubernetes**.
- Deploy **Wazuh** agents to monitor hosts, containers and the Kubernetes cluster at runtime (file integrity monitoring, log collection, rule-based threat detection).
- Centralize and visualize security events with the **Elastic Stack** (ELK: Elasticsearch, Logstash/Beats, Kibana).
- Use **TheHive** as the incident-response and case-management layer: qualified alerts are turned into structured, trackable cases with observables, tasks and a timeline.
- Demonstrate the complete scenario: vulnerable commit blocked by the pipeline, attack simulated against the deployed application, detection by Wazuh, visualization in Kibana, and case creation/investigation in TheHive.

1.4 Scope

In scope	SAST pipeline (Semgrep, SonarQube), Docker containerization, Kubernetes deployment, Wazuh runtime monitoring, ELK log analytics, TheHive incident management, documentation and demonstration.
Optional (if time permits)	Wazuh–TheHive automatic case creation via API/webhook (Cortex or custom connector), Kubernetes admission control, Grafana dashboards on top of ELK metrics.
Out of scope	Penetration testing of third-party systems, paid proprietary SIEM/SOAR licenses, formal regulatory compliance (ISO 27001, PCI-DSS audit).

2 Project Overview

A developer pushes code to the Git repository. The CI/CD pipeline is triggered automatically and takes the code through the following stages: **Code** → **Build** → **SAST** → **Deploy** (Docker/Kubernetes) → **Runtime monitoring** (Wazuh) → **Log analytics** (ELK) → **Incident response** (TheHive).

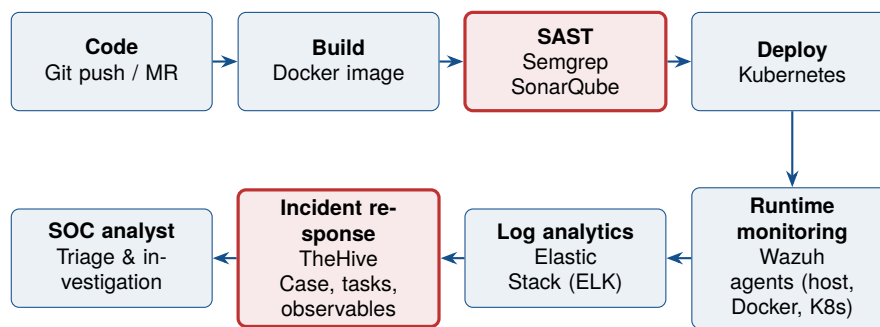


Figure 1: Platform overview: shift-left pipeline (top) feeding into runtime detection and response (bottom)

Guiding principle

Security is enforced twice: *before* deployment, through static analysis gates that keep known-bad code out of production, and *after* deployment, through continuous monitoring that assumes some risk will always reach production and must be detected quickly. The platform's value is the link between the two: every stage produces structured data that flows into the next one, from a Semgrep finding to a Kibana dashboard to a TheHive case.

3 Requirements Analysis

3.1 Actors

- **Developer:** pushes code and fixes vulnerabilities flagged by Semgrep/SonarQube.
- **DevSecOps / Platform engineer:** maintains the pipeline, the Docker images and the Kubernetes manifests.
- **SOC analyst (blue team):** monitors Kibana dashboards, tunes Wazuh rules and triages TheHive cases.

- **Incident responder:** investigates and closes cases in TheHive, documents the response.
- **Evaluator / supervisor:** reviews the documentation, the report and the demonstration.

3.2 Functional Requirements

Priorities: **M** = must have, **S** = should have, **C** = could have (optional).

Ref.	Description	Prio.
FR1	Provide a target application with documented, intentional vulnerabilities.	M
FR2	Automatically trigger the pipeline on every <i>push</i> or <i>merge request</i> .	M
FR3	Static application security testing of the source code with Semgrep.	M
FR4	Code-quality and security-quality gate with SonarQube (bugs, code smells, security hotspots).	M
FR5	Build the application into a Docker image as part of the pipeline.	M
FR6	Deploy the containerized application on Kubernetes.	M
FR7	Fail the pipeline (quality gate) when Semgrep/SonarQube findings exceed a defined severity threshold.	M
FR8	Deploy Wazuh agents on the hosts/containers to collect logs and monitor file integrity.	M
FR9	Forward Wazuh alerts and application logs into the Elastic Stack.	M
FR10	Provide Kibana dashboards for security events (failed logins, file changes, rule matches).	M
FR11	Create and track incidents in TheHive (case, observables, tasks, timeline) from qualified alerts.	M
FR12	Detect attack patterns via rules on logs (brute force, privilege escalation attempts, IDOR-like access patterns).	S
FR13	Automate alert-to-case creation from Wazuh/ELK into TheHive via API or connector.	C

Table 1: Functional requirements

3.3 Non-Functional Requirements

- **Reproducibility:** the whole platform (pipeline, Dockerfiles, Kubernetes manifests, Wazuh/ELK/TheHive configuration) must be re-runnable from the repository.
- **Traceability:** every case in TheHive must reference the originating alert (Wazuh rule ID, log source, timestamp).
- **Modularity:** each tool (Semgrep, SonarQube, Wazuh, ELK, TheHive) is loosely coupled and replaceable without rewriting the others.
- **Performance:** reasonable pipeline duration for repeated use; near-real-time propagation of alerts from Wazuh to Kibana.
- **Quality of results:** limit false positives and alert noise in both the SAST gate and the Wazuh rule set.
- **Lab security:** the vulnerable application and monitoring stack remain isolated and are never exposed to the Internet.
- **Documentation:** public repository with a clear README, architecture diagram and demonstration.

4 Technical Architecture

4.1 Pipeline and Security Operations Stages

Stage	Tool	Purpose	Blocking / escalation condition
SAST	Semgrep	Pattern-based static analysis: SQL injection, XSS, command injection, weak cryptography, dangerous functions	Pipeline fails on findings above the configured severity
Code quality & security gate	SonarQube	Bugs, code smells, vulnerabilities, security hotspots, duplication, coverage	Quality gate failure blocks the merge
Build & deploy	Docker, Kubernetes	Containerize and orchestrate the application	Build failure blocks deployment
Runtime monitoring	Wazuh	Host/container log collection, file integrity monitoring, rule-based detection	Rule match generates an alert
Log analytics	Elastic Stack (ELK)	Central storage, search and visualization of logs and alerts	Dashboard thresholds / saved searches trigger review
Incident response	TheHive	Case management: observables, tasks, timeline, collaboration	Alert qualified as incident → case opened

Table 2: Details of the pipeline and security operations stages

4.2 Environments

- **Development:** local machine with Docker.
- **Continuous integration:** Git-based CI/CD runner executing Semgrep, SonarQube scanner and the Docker build.
- **Runtime / staging:** Kubernetes cluster (kind or minikube) hosting the application, with Wazuh agents deployed alongside.
- **Security operations:** separate Elastic Stack and TheHive instances (own containers/namespace) receiving data from Wazuh, isolated from the application namespace.

5 Detailed Description of the Modules

5.1 Target Application

An intentionally vulnerable web application (authentication, REST API, database-backed), containerized with Docker, used both to trigger SAST findings in the pipeline and to generate realistic attack traffic for the monitoring stack to detect.

Illustrative vulnerabilities and where each is expected to surface:

Vulnerability	Detected by	Comment
SQL injection	Semgrep (SAST)	Pattern-matched in source code before deployment
XSS	Semgrep (SAST)	Pattern-matched in source code
Hard-coded secrets / weak crypto	Semgrep + SonarQube security hotspots	Flagged at the code-review stage
Brute-force login	Wazuh rule on authentication logs	Detected at runtime, visualized in Kibana
Unauthorized file/config changes	Wazuh file integrity monitoring	Detected at runtime
Suspicious access pattern (IDOR-like)	Wazuh custom rule on application/API logs	Requires an application-specific log format

Table 3: Vulnerabilities and their detection point in the pipeline

Scope decision

For a realistic first milestone, the minimum goal is to have Semgrep and SonarQube each catch at least one class of code-level vulnerability, and Wazuh detect at least two runtime attack patterns (e.g. brute force and file integrity violation), before extending the rule sets further.

5.2 End-to-End Scenario

The demonstration scenario connects the pre-deployment gate and the post-deployment detection loop:

Vulnerable commit → Semgrep/SonarQube finding → Pipeline blocked → Developer fix → Pipeline passes → Deployment on Kubernetes → Simulated attack → Wazuh alert → Kibana dashboard → TheHive case → Investigation & closure

This scenario will be documented as a detailed *before/after* walkthrough, covering both a blocked pipeline (shift-left) and a detected-and-investigated runtime attack (SOC loop).

5.3 Pipeline and Security Modules

5.3.1 SAST — Semgrep

Static analysis of the source code to detect SQL injection, XSS, command injection, insecure cryptography and dangerous functions using configurable rule sets (p/ci, p/owasp-top-ten, custom rules). Runs as a pipeline stage; results exported as JSON/SARIF and used as a pass/fail gate.

5.3.2 Code Quality & Security Gate — SonarQube

SonarQube scans the codebase for bugs, code smells, vulnerabilities and security hotspots, and computes a *Quality Gate* status. The pipeline stage fails the build when the Quality Gate is not passed (configurable conditions: new vulnerabilities, coverage on new code, duplicated lines).

5.3.3 Containerization — Docker

The application is packaged as a Docker image built in the pipeline. Base images are pinned and kept minimal to reduce the attack surface; the image is then pushed to a registry for deployment.

5.3.4 Orchestration — Kubernetes

The Docker image is deployed on a Kubernetes cluster through versioned manifests (Deployment, Service, ConfigMap/Secret). Kubernetes provides the runtime environment monitored by Wazuh.

5.3.5 Runtime Security Monitoring — Wazuh

Wazuh agents are deployed on the hosts/containers to provide:

- log collection and rule-based analysis (authentication failures, privilege escalation attempts);
- file integrity monitoring (FIM) on sensitive paths and configuration files;
- vulnerability detection on installed packages;
- alert generation forwarded to the Wazuh indexer and onward to the Elastic Stack.

5.3.6 Log Analytics — Elastic Stack (ELK)

Elasticsearch stores and indexes logs and Wazuh alerts; Logstash/Beats handle ingestion and parsing; Kibana provides dashboards for the SOC analyst (alert volume over time, top rule IDs, source IPs, affected assets).

5.3.7 Incident Response — TheHive

Alerts qualified as significant (manually or, optionally, automatically through an API/webhook connector) are turned into **cases** in TheHive, each with:

- observables (IP addresses, usernames, file hashes, URLs) extracted from the Wazuh alert;
- tasks representing the investigation steps (containment, eradication, lessons learned);
- a timeline documenting the analyst's actions and findings;
- a severity, tags and a final resolution status (true positive / false positive / closed).

5.4 Alert-to-Case Workflow

Instead of a purely CVE-driven risk score, the platform's operational logic is an **alert qualification workflow**:

1. A Wazuh rule matches an event (e.g. rule level ≥ 10 , or a specific rule ID such as repeated authentication failures).
2. The alert is indexed in Elasticsearch and visible in a Kibana dashboard in near real time.
3. The SOC analyst reviews the alert. If it is judged actionable, a case is created in TheHive (manually via the UI, or automatically via the TheHive API/Cortex connector for pre-defined rule IDs).
4. The case is enriched with observables pulled from the alert (source IP, username, affected host) and assigned tasks.
5. The analyst investigates, documents findings in the case timeline, and closes the case with a resolution status.

Rule-to-severity mapping

Wazuh rule levels are mapped to a case priority in TheHive (for example: level $\geq 12 \rightarrow$ high priority case opened automatically; level 7–11 $\rightarrow$ alert surfaced in Kibana for manual review; level $< 7 \rightarrow$ logged only). This mapping must be documented and adjusted during the project to control alert volume and noise.

6 Technology Stack

Area	Technology	Status
Backend / target app	Language of choice (Python/Node.js)	Selected
Containers	Docker	Selected
Orchestration	Kubernetes	Selected
Version control & CI/CD	Git + CI runner	Selected
SAST	Semgrep	Selected
Code quality / security gate	SonarQube	Selected
Runtime monitoring / HIDS	Wazuh	Selected
Log storage, search, dashboards	Elastic Stack (Elasticsearch, Logstash/Beats, Kibana)	Selected
Incident response / case management	TheHive	Selected
Alert-to-case automation	Cortex or custom TheHive API integration	Optional

Table 4: Project technologies

7 Dependencies

7.1 Dependencies Between Modules

Module	Depends on
CI/CD pipeline	Git repository, CI runner, target application source
SonarQube quality gate	SonarQube server reachable from the CI runner
Docker build & deploy	Docker registry, Kubernetes cluster credentials
Wazuh monitoring	Wazuh manager, agents installed on hosts/containers, network access from agents to manager
Elastic Stack	Wazuh indexer output (or Filebeat/Wazuh module), Elasticsearch cluster reachable by Kibana
TheHive	Case templates, analyst accounts, (optional) API token for automated case creation from Wazuh/ELK

Table 5: Internal dependencies

7.2 External Dependencies and Prerequisites

- **Infrastructure:** Docker, a local Kubernetes cluster (kind or minikube), a Git server with CI runner.
- **Security tools:** Semgrep, SonarQube (server + scanner), Wazuh (manager + agents + indexer), Elastic Stack, TheHive (versions pinned for reproducibility).
- **Skills:** application security basics, Docker/Kubernetes, CI/CD, log analysis, incident-response fundamentals.

8 Planning

Estimated duration: 14 to 16 weeks.

Week	Tasks	Deliverable
W1-2	Scoping, repository setup, target application with a few intentional vulnerabilities, Dockerization	Dockerized application
W3-4	Pipeline with Semgrep and SonarQube quality gate	SAST pipeline
W5-6	Kubernetes manifests and deployment of the application	Running deployment on K8s
W7-8	Wazuh manager/agents deployment, first detection rules	Runtime monitoring in place
W9-10	Elastic Stack integration, Kibana dashboards, TheHive setup and case templates	SOC visualization & case management
W11-12	End-to-end scenario, alert-to-case workflow, documentation, demo video	Public repository and report

Table 6: Provisional schedule

Extensions if time permits: automated Wazuh-to-TheHive case creation, Kubernetes admission control, Grafana dashboards.

9 Deliverables

- Source code of the target application.
- CI/CD pipeline configuration (Semgrep, SonarQube stages) and associated scripts.
- Dockerfile(s) and Kubernetes manifests.
- Wazuh rules/configuration used for detection.
- Kibana dashboards (exported) and TheHive case templates.
- Public repository with a clear README and architecture diagram.
- Short demonstration video or GIF.
- Written report including the before/after study: a blocked pipeline run and a detected/investigated runtime attack.

10 Validation Criteria and Indicators

- Detection rate on the intentionally injected code-level vulnerabilities (Semgrep/SonarQube).
- Number of false positives and noise reduction achieved by tuning the Wazuh rule set.
- Pipeline execution time.
- Effective blocking of a merge/deployment containing a flagged vulnerability, then success after the fix.
- Time from simulated attack to alert visible in Kibana, and from alert to case opened in TheHive.
- Reproducibility: a third party can re-run the platform by following the README.

11 Risks and Mitigation Measures

Risk	Mitigation measure
Too many tools to integrate well in the available time	Get the core chain (Semgrep → SonarQube → Docker/K8s → Wazuh → ELK → TheHive) working end-to-end first, defer automation
Alert fatigue / noisy Wazuh rules	Start with a small, high-confidence rule set; tune iteratively; document exceptions
SonarQube/Wazuh resource footprint on a laptop-scale lab	Reduce retention, limit indices, run components sequentially if needed
Manual alert-to-case step slows the demo	Keep a documented manual procedure as the default; automation is optional
Pipeline too slow	Cache dependencies, run Semgrep/SonarQube in parallel where possible
Overlap with generic CI/CD security tutorials	Emphasize the runtime detection and incident-response loop (Wazuh/ELK/TheHive), not just the pipeline gate

Table 7: Risk analysis

12 Ethical and Legal Constraints

- Simulated attacks target only the lab's own application, hosted in an isolated environment.

- No scan or attack is carried out against third-party systems without authorization.
- The vulnerable application and monitoring stack are never publicly exposed; demonstration credentials/secrets are fictitious.
- The tools used are open source or have a license suited to academic use.

13 Conclusion

This project connects application security, containerized delivery and security operations within a single platform. Its value rests on the continuity between shift-left gating (Semgrep, SonarQube) and shift-right detection and response (Wazuh, ELK, TheHive), demonstrated end-to-end on an application deployed with Docker and Kubernetes. This specification document defines the scope, architecture, dependencies and schedule required to carry it out.

A Appendix A — Indicative CI/CD Pipeline Skeleton

Schematic skeleton to be adapted (job names and options to be validated during implementation):

```
stages:
  - sast
  - quality-gate
  - build
  - deploy

semgrep:
  stage: sast
  script: semgrep --config p/owasp-top-ten --json -o semgrep.json .

sonarqube:
  stage: quality-gate
  script: sonar-scanner -Dsonar.qualitygate.wait=true

docker-build:
  stage: build
  script:
    - docker build -t $IMAGE .
    - docker push $IMAGE

deploy:
  stage: deploy
  script: kubectl apply -f k8s/
```

B Appendix B — Wazuh Alert to TheHive Case (Pseudo-code)

```
# Simplified connector: poll Wazuh alerts, open a TheHive case
# for alerts at or above a configured rule level.

RULE_LEVEL_THRESHOLD = 12

def poll_wazuh_alerts():
    return wazuh_api.get_alerts(since=last_check_time)

def to_observables(alert):
    return [
        {"dataType": "ip", "data": alert["srcip"]},
```

```
        {"dataType": "other", "data": alert["rule_id"]},
    ]

def handle_alert(alert):
    if alert["rule_level"] >= RULE_LEVEL_THRESHOLD:
        thehive_api.create_case(
            title=f"Wazuh alert: {alert['rule_description']}",
            severity=map_level_to_severity(alert["rule_level"]),
            observables=to_observables(alert),
            tags=["wazuh", alert["rule_group"]],
        )

def run():
    for alert in poll_wazuh_alerts():
        handle_alert(alert)
```